COER UNIVERSITY, ROORKEE

Department of Computer Science & Engineering

Full Stack Web Development

Unit II Assignment: Front-End Development — HTML, CSS & JavaScript

Name: Arpit Giri	**CU ID:** CU24250181
Roll No.: 11	**Subject:** Full Stack Web Development (BTCS303T)

Question 1. Write CSS Flexbox code for a navbar with a logo on the left and 3 menu links on the right, all vertically centered.

Answer:

HTML:

```
<nav class="navbar">
  <div class="logo">MyLogo</div>
  <ul class="nav-links">
    <li><a href="#home">Home</a></li>
    <li><a href="#about">About</a></li>
    <li><a href="#contact">Contact</a></li>
  </ul>
</nav>
```

CSS (Flexbox):

```
* { box-sizing: border-box; }

.navbar {
  display: flex;
  justify-content: space-between;   /* logo -> far left, links -> far right */
  align-items: center;              /* vertical centering of everything */
  height: 64px;
  padding: 0 24px;
  background: #1f3a5f;
}

.logo {
  color: #fff;
  font-size: 1.5rem;
  font-weight: bold;
}

.nav-links {
  display: flex;                    /* puts the 3 links in a row */
  align-items: center;
  gap: 24px;
  margin: 0;
  padding: 0;
  list-style: none;
}

.nav-links a {
  color: #fff;
  text-decoration: none;
}
```

Question 2. This button is supposed to toggle a "dark-mode" class on the body when clicked, but nothing happens. Find and fix the bug.

```
const button = document.querySelector('.toggle-btn');
button.addEventListner('click', () => {
  document.body.classList.toggle('dark-mode');
});
```

Answer:

Bug: The method name is misspelled - `addEventListner` (missing an **e**). The correct name is `addEventListener`. Since `addEventListner` does not exist, JavaScript throws *TypeError: button.addEventListner is not a function*, so the click handler is never attached.

Fixed code:

```
const button = document.querySelector('.toggle-btn');
button.addEventListener('click', () => {
  document.body.classList.toggle('dark-mode');
});
```

Tip: also make sure the script runs after the button exists (place the `<script>` at the end of `<body>` or use the `defer` attribute), otherwise `querySelector` returns null.

Question 3. Write vanilla JavaScript to validate a signup form: check that the email contains "@" and the password is at least 8 characters, before allowing submission - without using any library.

Answer:

HTML:

```
<form id="signupForm" novalidate>
  <input type="email" id="email" name="email" placeholder="Email">
  <span id="emailError" class="error"></span>

  <input type="password" id="password" name="password" placeholder="Password">
  <span id="passwordError" class="error"></span>

  <button type="submit">Sign Up</button>
</form>
```

JavaScript:

```
const form = document.getElementById('signupForm');
const emailInput = document.getElementById('email');
const passwordInput = document.getElementById('password');
const emailError = document.getElementById('emailError');
const passwordError = document.getElementById('passwordError');

form.addEventListener('submit', function (e) {
  let isValid = true;
  emailError.textContent = '';
  passwordError.textContent = '';

  const email = emailInput.value.trim();
  const password = passwordInput.value;

  if (!email.includes('@')) {
    emailError.textContent = 'Email must contain "@"';
    isValid = false;
  }

  if (password.length < 8) {
    passwordError.textContent = 'Password must be at least 8 characters';
    isValid = false;
  }

  if (!isValid) {
    e.preventDefault();          // block submission when validation fails
  }
  // if isValid is true, the form is submitted normally
});
```

Question 4. This code attaches a new scroll event listener every time renderPage() runs, without ever removing the old one. In 2 lines, explain why this causes a memory leak, then rewrite it correctly.

```
function renderPage() {
  window.addEventListener('scroll', () => {
    console.log(window.scrollY);
  });
}
renderPage();
renderPage();
```

Answer:

Explanation (2 lines):

1. Each call adds a brand-new anonymous function, so listeners (and everything they capture) pile up on *window* and can never be garbage-collected.

2. Every scroll then runs all of them, so memory use keeps growing and scrolling gets slower and logs duplicates.

Correct code:

```
function onScroll() {                        // ONE named function, defined once
  console.log(window.scrollY);
}

function renderPage() {
  window.removeEventListener('scroll', onScroll);  // remove old one first (safe if none)
  window.addEventListener('scroll', onScroll);     // same reference -> only one listener
}

renderPage();
renderPage();      // still only ONE scroll listener is active

// When the page/view is destroyed, clean up:
// window.removeEventListener('scroll', onScroll);
```

Question 5. Write a debounced search-input handler in JavaScript that waits 400ms after the user stops typing before calling an API function searchProducts(query).

Answer:

```
// Generic debounce helper
function debounce(fn, delay) {
  let timer;                                  // remembers the pending timeout
  return function (...args) {
    clearTimeout(timer);                      // every keystroke cancels the previous timer
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}

const searchInput = document.getElementById('search');

const handleSearch = debounce(function (event) {
  const query = event.target.value.trim();
  if (query !== '') {
    searchProducts(query);                    // runs only after 400ms of no typing
  }
}, 400);

searchInput.addEventListener('input', handleSearch);
```

How it works: each keystroke clears the running timer and starts a new 400ms one, so *searchProducts()* is called only once - after the user pauses - instead of on every key press.

Question 6. This layout breaks below 480px screen width. Add a media query that fixes it for mobile, and explain in 1 line why the original CSS breaks.

```
.container {
  display: flex;
  width: 960px;
}
.card {
  width: 300px;
}
```

Answer:

Why it breaks (1 line): The container has a fixed width of 960px, which is far wider than a screen under 480px, so the content overflows and forces horizontal scrolling.

Fixed CSS:

```
.container {
  display: flex;
  width: 960px;
}
.card {
  width: 300px;
}

/* Mobile fix */
@media (max-width: 480px) {
  .container {
    flex-direction: column;   /* stack cards vertically */
    width: 100%;              /* fit the screen instead of fixed 960px */
  }
  .card {
    width: 100%;              /* each card fills the screen width */
  }
}
```

Make sure the HTML contains `<meta name="viewport" content="width=device-width, initial-scale=1">` so the media query works on real phones.

Question 7. Write JavaScript code using localStorage to save and restore a user's selected theme ("light" or "dark") so it persists across page reloads.

Answer:

HTML + CSS:

```
<button id="themeToggle">Toggle Theme</button>

<style>
  body.light { background: #ffffff; color: #111111; }
  body.dark  { background: #121212; color: #eeeeee; }
</style>
```

JavaScript:

```
const toggleBtn = document.getElementById('themeToggle');

function applyTheme(theme) {
  document.body.classList.remove('light', 'dark');
  document.body.classList.add(theme);
  toggleBtn.textContent = theme === 'dark' ? 'Switch to Light' : 'Switch to Dark';
}

// RESTORE: runs on every page load/reload
const savedTheme = localStorage.getItem('theme');
applyTheme(savedTheme === 'dark' ? 'dark' : 'light');   // default = light

// SAVE: runs when the user changes the theme
toggleBtn.addEventListener('click', () => {
  const newTheme = document.body.classList.contains('dark') ? 'light' : 'dark';
  applyTheme(newTheme);
  localStorage.setItem('theme', newTheme);              // persists after reload
});
```

Question 8. This form refreshes the page on submit and loses all JS state. Rewrite the code to prevent the default behavior and submit the data using fetch() instead.

```
<form id="contactForm">
  <input name="email" />
  <button type="submit">Send</button>
</form>
```

Answer:

Rewritten code:

```
<form id="contactForm">
  <input name="email" />
  <button type="submit">Send</button>
</form>

<script>
  const form = document.getElementById('contactForm');

  form.addEventListener('submit', async (e) => {
    e.preventDefault();                          // stops the page refresh

    const data = Object.fromEntries(new FormData(form));   // { email: "..." }

    try {
      const res = await fetch('/api/contact', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(data)
      });

      if (!res.ok) throw new Error('Server error: ' + res.status);

      const result = await res.json();
      console.log('Success:', result);
      form.reset();                              // clear the input after sending
    } catch (err) {
      console.error('Submit failed:', err);
    }
  });
</script>
```

Question 9. This async function is meant to return the parsed user data, but callers receive a pending Promise instead. Identify the missing keyword and correct the code.

```
async function getUser(id) {
  const res = fetch(`/api/users/${id}`);
  const data = res.json();
  return data;
}
```

Answer:

Missing keyword: `await`. Both `fetch()` and `res.json()` return Promises, so each needs `await`. Without it, `res` is a Promise (not a Response object), so the function does not produce the parsed data.

Corrected code:

```
async function getUser(id) {
  const res = await fetch(`/api/users/${id}`);   // wait for the HTTP response
  if (!res.ok) throw new Error(`HTTP error ${res.status}`);
  const data = await res.json();                 // wait for the body to be parsed
  return data;
}

// Callers must also wait, because an async function always returns a Promise:
const user = await getUser(5);                   // inside another async function
// or
getUser(5).then(user => console.log(user));
```

Question 10. Write CSS Grid code for a responsive image gallery with 4 columns on desktop and 2 columns on mobile, using minmax() and auto-fit instead of a media query.

Answer:

HTML:

```
<div class="gallery">
  <img src="img1.jpg" alt="Photo 1">
  <img src="img2.jpg" alt="Photo 2">
  <img src="img3.jpg" alt="Photo 3">
  <img src="img4.jpg" alt="Photo 4">
  <!-- ...more images -->
</div>
```

CSS Grid (no media query):

```
.gallery {
  --gap: 16px;
  --min: 150px;          /* smallest column width -> gives 2 columns on phones */

  display: grid;
  gap: var(--gap);
  grid-template-columns: repeat(
    auto-fit,
    minmax(max(var(--min), calc((100% - 3 * var(--gap)) / 4)), 1fr)
  );
}

.gallery img {
  width: 100%;
  aspect-ratio: 1 / 1;
  object-fit: cover;
  display: block;
  border-radius: 8px;
}
```

How it works: `auto-fit` fits as many columns as possible; `minmax(min, 1fr)` sets each column's minimum width and lets it stretch to fill the row. The minimum is the larger of 150px and one quarter of the container, so wide screens are capped at **4 columns**, while on a phone (about 320-480px) only two 150px columns fit, giving **2 columns**. Tablet widths naturally show 3.

Simpler version: `grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));` also works, but it keeps adding columns on very wide screens, so it needs a `max-width` on the container to stay at 4.